

From data to figure

Here I present my annotations to the original code Ben shared with me (`moments_script_STANCE.py`).

Note. Tips for readability

PEP 8, the official *Python Style Guide*, recommends:

- **Limit all lines to a maximum of 79 characters** for code.
- **Limit docstrings and comments to 72 characters.**

These days, people use wider monitors, so teams relax this rule to **88 characters**. So going over *79 characters* is not a dealbreaker *per se*, but it's also a good idea to limit your codes and comments rather *short*.

Note2. How to read this notebook.

Section 1 discusses how to read files with multiple headers using `MultiIndex` feature. If you read motion capture output data, section 1 may be useful.

Section 2 continues discussing the use of `pd.IndexSlice`, a useful method to navigate a DataFrame with MultiIndex.

Section 3 shows how to improve your custom function. I tried to introduce what's discussed in Week 6 (ex. docstring), but also share how I would handle different tasks.

Section 4 continues discussing how to *modularize* procedures by making utility functions.

Section 5 describes how to visualize multiple datasets using a for loop.

You can skip sections 2, 3, and 4 if you don't find them too relevant. Also, the original code is somewhat bulky to read, so only reading through my annotation could be enough for your understanding.

1. Reading a file with complex structure

The code reads an `.xls` file (`Shefaali_Walk1_Baseline_Data.xls`) that appears to be an output of a motion capture system.

I usually open a file before writing codes. This is to figure out how to read a file. For example, the first row (1) repeatedly stores the path where a `.c3d` file is saved. I can skip it using `skiprows`.

Out[1]:	Unnamed: 0	RHS	RTO	LHS	LTO
0	NaN	EVENT_LABEL	EVENT_LABEL	EVENT_LABEL	EVENT_LABEL
1	NaN	ORIGINAL	ORIGINAL	ORIGINAL	ORIGINAL
2	ITEM	X	X	X	X
3	1	0.24	0.92	0.79	0.36
4	2	1.31	2.01	1.88	1.45

Good. It's read fine. The first column is 'Unnamed: 0', because in the `.xls` file, it's a blank cell. In fact, the entire column (2) is just indices of the values. This we can skip as well.

However, if you look closely, the first four rows (3) in each column would together make a unique column header. Can we make use of that?

```
In [2]: # When you give what rows to use as headers,
# you can provide a list of row indices.
# Here, we want the first four rows (0, 1, 2, 3) to be used
# as headers.
# Remember, this will prepare multiIndex (discussed in Week 3)
data = pd.read_excel('Shefaali_Walk1_Baseline_Data.xls', skiprows=1,
                    header=[0, 1, 2, 3])

# You can then drop the first column, which is just indices.
# Using `data.columns[0]` gets the name of the first column.
data = data.drop(columns=data.columns[0])
data.head()
```

```
Out[2]:
```

	RHS	RTO	LHS	LTO	Bad
	EVENT_LABEL	EVENT_LABEL	EVENT_LABEL	EVENT_LABEL	EVENT_LABEL
	ORIGINAL	ORIGINAL	ORIGINAL	ORIGINAL	ORIGINAL
	X	X	X	X	0
0	0.24	0.92	0.79	0.36	NaN
1	1.31	2.01	1.88	1.45	NaN
2	2.40	3.05	2.92	2.54	NaN
3	3.44	4.12	3.98	3.57	NaN
4	4.51	5.21	5.08	4.64	NaN

5 rows x 37 columns

MultiIndex

In week 3 (`data_wrangling_pandas_part2.py`), `MultiIndex` columns were briefly introduced. There we concatenated strings to make simple column names. Here we will go ahead and use `MultiIndex` as is.

```
In [3]: # See what a MultiIndex column looks like.
data.columns[0:5]
```

```
Out[3]: MultiIndex([('RHS', 'EVENT_LABEL', 'ORIGINAL', 'X'),
                    ('RTO', 'EVENT_LABEL', 'ORIGINAL', 'X'),
                    ('LHS', 'EVENT_LABEL', 'ORIGINAL', 'X'),
                    ('LTO', 'EVENT_LABEL', 'ORIGINAL', 'X'),
                    ('Bad', 'EVENT_LABEL', 'ORIGINAL', 0)],
                    )
```

MultiIndex has levels. The first element of a tuple will be the highest level.

`.get_level_valus(level)` will return strings that correspond to the specified `level`.

```
In [4]: # If you want the highest level labels (level=0):
data.columns.get_level_values(0)
```

```
Out[4]: Index(['RHS', 'RT0', 'LHS', 'LT0', 'Bad', 'FP1', 'FP1', 'FP1', 'FP2',
'FP2',
'FP2', 'FP1', 'FP1', 'FP1', 'FP2', 'FP2', 'FP2', 'Right Ankle
Angle',
'Right Knee Angle', 'Right Hip Angle', 'Left Ankle Angle',
'Left Knee Angle', 'Left Hip Angle', 'Right Ankle Moment',
'Right Knee Moment', 'Right Hip Moment', 'Left Ankle Moment',
'Left Knee Moment', 'Left Hip Moment', 'Right Ankle Power',
'Right Knee Power', 'Right Hip Power', 'Left Ankle Power',
'Left Knee Power', 'Left Hip Power', 'CGVel', 'CGVel'],
dtype='object')
```

In order to access a column, you need to provide a full tuple.

```
In [5]: # First five rows of the column: ('FP1', 'FORCE', 'ORIGINAL', 'X')
data.loc[0:5, ('FP1', 'FORCE', 'ORIGINAL', 'X')]
```

```
Out[5]: 0    0.0
1    0.0
2    0.0
3    0.0
4    0.0
5    0.0
Name: (FP1, FORCE, ORIGINAL, X), dtype: float64
```

`pd.IndexSlice` can help you with flexible indexing

This way you can stop at a certain level and return all lower level columns altogether.

```
In [6]: # Only provide part of the tuple.
data.loc[:, pd.IndexSlice['FP1', 'FORCE']].head()
```

```
/var/folders/b4/_bb0snyn7nn6f7d4sk8ry5jr0000gn/T/ipykernel_59657/362587
8034.py:2: PerformanceWarning: indexing past lexsort depth may impact p
erformance.
data.loc[:, pd.IndexSlice['FP1', 'FORCE']].head()
```

Out [6]:

	ORIGINAL		
	X	Y	Z
0	0.0	0.0	0.0
1	0.0	0.0	0.0
2	0.0	0.0	0.0
3	0.0	0.0	0.0
4	0.0	0.0	0.0

```
In [7]: # PerformanceWarning will go away if you sort the columns first.
# This will sort the columns based on the MultiIndex,
# so if you want to keep the original column order,
# you can skip this step. PerformanceWarning is just a warning.
data = data.sort_index(axis=1)
data.loc[:, pd.IndexSlice['FP1', 'FORCE']].head()
```

Out [7]:

	ORIGINAL		
	X	Y	Z
0	0.0	0.0	0.0
1	0.0	0.0	0.0
2	0.0	0.0	0.0
3	0.0	0.0	0.0
4	0.0	0.0	0.0

Comparison with the original method I

Ben's original code (lines 1-83) proceeds in this manner:

1. read the file repeatedly to save column headers at different levels to different variables.
2. read the excel file skipping 5 rows and assign headers of choice

```
In [8]: # Adapted from the original code for conciseness.
column_names = pd.read_excel('Shefaali_Walk1_Baseline_Data.xls',
                             header=None, nrows=2)
footcolumn_names = pd.read_excel('Shefaali_Walk1_Baseline_Data.xls',
                                 header=None, nrows=5)
true_headers = column_names.iloc[1]
foot_labels = footcolumn_names.iloc[3]
df = pd.read_excel('Shefaali_Walk1_Baseline_Data.xls',
                   header=None, skiprows=5)
```

```
df.columns = true_headers
df.head()
```

Out[8]:

	1	NaN	RHS	RTO	LHS	LTO	Bad	FP1	FP1	FP1	FP2	...	Left Knee Moment	L M
0	1	0.24	0.92	0.79	0.36	NaN	0.0	0.0	0.0	29.17310	...	NaN		
1	2	1.31	2.01	1.88	1.45	NaN	0.0	0.0	0.0	28.59716	...	NaN		
2	3	2.40	3.05	2.92	2.54	NaN	0.0	0.0	0.0	28.01432	...	-0.01031	-(	
3	4	3.44	4.12	3.98	3.57	NaN	0.0	0.0	0.0	27.43740	...	-0.01316	-(	
4	5	4.51	5.21	5.08	4.64	NaN	0.0	0.0	0.0	26.87540	...	-0.03113	-(	

5 rows x 38 columns

This can be one way, but we see that the first column's name is `NaN`, and there are duplicate names (`FP1`). This will be tricky, because there's no other information provided other than the names, so you don't know which column corresponds to which axis.

```
In [9]: # There are six columns named 'FP1'.
# The first three are x, y, and z axis of FORCE/ORIGINAL.
# The next three are x, y, and z axis of COFP/ORIGINAL.

# Right now we can't tell which is which:
df['FP1'].head()
```

```
Out[9]:
```

	1	FP1	FP1	FP1	FP1	FP1	FP1
0	0.0	0.0	0.0	NaN	NaN	NaN	
1	0.0	0.0	0.0	NaN	NaN	NaN	
2	0.0	0.0	0.0	NaN	NaN	NaN	
3	0.0	0.0	0.0	NaN	NaN	NaN	
4	0.0	0.0	0.0	NaN	NaN	NaN	

```
In [10]: # Using MultiIndex columns can help.
# You now know which column is which.
data.loc[:, ('FP1')].head()
```

Out[10]:

	COFP			FORCE		
	ORIGINAL			ORIGINAL		
	X	Y	Z	X	Y	Z
0	NaN	NaN	NaN	0.0	0.0	0.0
1	NaN	NaN	NaN	0.0	0.0	0.0
2	NaN	NaN	NaN	0.0	0.0	0.0
3	NaN	NaN	NaN	0.0	0.0	0.0
4	NaN	NaN	NaN	0.0	0.0	0.0

2. Manipulate original columns to make new columns

Ben's original code (lines 96-125) then goes manipulate columns to create new columns: `Left_CGVel_Avg` and `Right_CGVel_Avg`.

Using my `data`, we can do it this way:

```
In [11]: data[('CGVel', 'KINETIC_KINEMATIC', 'LFT', 'Avg')] =\
          data.loc[:, pd.IndexSlice['CGVel', :, 'LFT', :]].mean(axis=1)
data[('CGVel', 'KINETIC_KINEMATIC', 'RFT', 'Avg')] =\
          data.loc[:, pd.IndexSlice['CGVel', :, 'RFT', :]].mean(axis=1)

# Show the resulting columns
data.loc[:, pd.IndexSlice['CGVel', :, :, 'Avg']].head()
```

Out[11]:

	CGVel	
	KINETIC_KINEMATIC	
	LFT	RFT
	Avg	Avg
0	NaN	NaN
1	NaN	NaN
2	-1.21126	2.51602
3	-1.22225	2.58026
4	-1.22255	2.62753

```
In [12]: # `pd.IndexSlice` helps select specific levels of MultiIndex.
          # The line below will select all columns
```

```
# where the first level is 'CGVel', the third level is 'LFT',
# and the other levels can be anything.
# ('Avg' is included because we ran the line above to create it.)
data.loc[:, pd.IndexSlice['CGVel', :, 'LFT', :]].head()
```

Out[12]: **CGVel**

KINETIC_KINEMATIC		
		LFT
	Y	Avg
0	NaN	NaN
1	NaN	NaN
2	-1.21126	-1.21126
3	-1.22225	-1.22225
4	-1.22255	-1.22255

In [13]: *# Of course, if you don't know what the second level header is,
you can always check the columns of the DataFrame.
We saw '.get_level_values(level)' can help with the task.*

```
# For example, first save all columns whose first level  
# labels are 'CGVel':
```

```
cgvel_multi_index = data.columns[
    data.columns.get_level_values(0) == 'CGVel']
```

```
# Check what 'cgvel_multi_index' looks like
cgvel_multi_index
```

Out[13]: MultiIndex([('CGVel', 'KINETIC_KINEMATIC', 'LFT', 'Y'),
('CGVel', 'KINETIC_KINEMATIC', 'RFT', 'Y'),
('CGVel', 'KINETIC_KINEMATIC', 'LFT', 'Avg'),
('CGVel', 'KINETIC_KINEMATIC', 'RFT', 'Avg')],
)

In [14]: *# We need to get 'KINETIC_KINEMATIC',
which is the second level header.*

```
# Since 'cgvel_multi_index' is a list of MultiIndex,  
# we can use '.get_level_values()' on it.  
# Set level=1 to get the second level values.
```

```
# '.unique()' gets unique values only.
second_level_headers = \
    cgvel_multi_index.get_level_values(1).unique()
```

```
# Check the result
second_level_headers
```



```
Out[14]: Index(['KINETIC_KINEMATIC'], dtype='object')
```

```
In [15]: # Then indexing this output using `[0]`
# gets the first (and only) value: 'KINETIC_KINEMATIC'
kinematic_header = second_level_headers[0]

# Check the result
kinematic_header

# Then you can use it in the code above:
# (Don't run lines below again; they are just for illustration.

# >>> idx = pd.IndexSlice
# >>> data[('CGVel', kinematic_header, 'LFT', 'Avg')] = \
#       (data.loc[:, idx['CGVel', kinematic_header, 'LFT', :]]
#       .mean(axis=1))
# >>> data[('CGVel', kinematic_header, 'RFT', 'Avg')] = \
#       (data.loc[:, idx['CGVel', kinematic_header, 'RFT', :]]
#       .mean(axis=1))
```

```
Out[15]: 'KINETIC_KINEMATIC'
```

Comparison with the original method II

This is how the original code works:

1. prepare empty lists, and append all columns with the header: `CGVel` if the column's `foot_label` either matches `"LFT"` or `"RFT"`.
2. calculate columnwise means each for `"LFT"` and `"RFT"`.

```
In [16]: # This is the current DataFrame loaded.
# We see that the two columns are indistinguishable
# because they have the same name 'CGVel'.
df.loc[:, df.columns == 'CGVel'].head()
```

```
Out[16]:
```

	1	CGVel	CGVel
0		NaN	NaN
1		NaN	NaN
2		-1.21126	2.51602
3		-1.22225	2.58026
4		-1.22255	2.62753

```
In [17]: # Ben's code (lines 96-125) goes like this:
left_cgvel_idx = []
right_cgvel_idx = []
```

```

for i, col in enumerate(df.columns):
    if "CGVEL" in str(col).upper():
        label = str(foot_labels.iloc[i]).strip().upper()
        if label == "LFT":
            left_cgvel_idxes.append(i)
        elif label == "RFT":
            right_cgvel_idxes.append(i)

if left_cgvel_idxes:
    df['Left_CGVel_Avg'] = df.iloc[:, left_cgvel_idxes].mean(axis=1)
    left_cgvel_col = "Left_CGVel_Avg"
else:
    left_cgvel_col = None

if right_cgvel_idxes:
    df['Right_CGVel_Avg'] = df.iloc[:, right_cgvel_idxes].mean(axis=1)
    right_cgvel_col = "Right_CGVel_Avg"
else:
    right_cgvel_col = None

# Check the output
df[['Left_CGVel_Avg', 'Right_CGVel_Avg']].head()

```

Out[17]:

	Left_CGVel_Avg	Right_CGVel_Avg
0	NaN	NaN
1	NaN	NaN
2	-1.21126	2.51602
3	-1.22225	2.58026
4	-1.22255	2.62753

	Left_CGVel_Avg	Right_CGVel_Avg
0	NaN	NaN
1	NaN	NaN
2	-1.21126	2.51602
3	-1.22225	2.58026
4	-1.22255	2.62753

This is also a valid method - you just need to do more mental juggling!

3. Helper functions

Functions are prepared in the original code:

1. `extract_peak_per_stride(joint_label, direction='max')` :
extract peak moments of a joint (ex. Right Knee) per stride
2. `to_frames(col_name)` :
convert gait events from seconds to frame indices
3. `average_middle_values(arr)` :

average middle values (this is not extensively used)

This section you can skip, if you don't find too relevant.

What can be improved?

1. **Docstring:** I suggest using docstring. Also, keep in mind of the style recommendation of **PEP 8**.
2. **local variables vs. global variables:** `extract_peak_per_stride` uses variables defined **outside** (ex. `moment_columns` or `df`). If you accidentally erase these variables or change the names of them, this function would not operate until you reflect such changes. This is a risky behavior. I recommend minimizing the use of variables defined outside a function.
3. **foolproof indexing:** use `.iloc` if you use indices.
4. **consistent output type:** if you return `numpy.ndarray` for one, do the same for another.

```
# %%==== Define Helper Function to extract peak moments per stride ====
def extract_peak_per_stride(joint_label, direction='max'): 1
    col_name = moment_columns[joint_label]                #defines a function that pulls out peak mom
    data_series = df[col_name]                             #col_name is the moment column name with a
    starts, ends = (lhs_frames, lto_frames) if "Left" in joint_label else (rhs_frames, rto_frames) 2
    peak_values = []                                       #creates a data frame with the joint moment
    for start in starts:                                  #creates an empty data frame for peak value
        end_candidates = ends[ends > start]                #indexes starts as the first start in the l
        if len(end_candidates) == 0:                       #creates a list of candidates for the end v
            continue                                       #if the value is the same (so the distance
        end = end_candidates[0]                            #remember zero indexing; the end is the fir

        # Skip bad segments
        cycle_frame_range = set(range(start, end))         #defines the gait cycle range from previous
        if not cycle_frame_range.isdisjoint(bad_frame_indices): 3
            continue                                       # Only keep going if cycle_frame_range does

        segment = data_series[start:end].values            #each gait cycle is referred to as "segment"
        time_segment = time_seconds[start:end]             #this is used for the impulse, not peak mom

        if len(segment) < 5:                               #helps filter out noise in case the length
            continue                                       #direction of the peak will be defined late

        if direction == 'max':                             #otherwise, pull the minimum value by using
            peak_val = np.max(segment)
        elif direction == 'min':
            peak_val = np.min(segment)
        else:
            raise ValueError("Direction must be 'max' or 'min'") #or if it's not defined, raise an error th

        peak_values.append(peak_val)                       #the peak values that are pulled by this if

    if peak_values:
        return np.mean(peak_values), peak_values          4
    else:
        return np.nan, []                                  #if the peak_values variable is called, pro
                                                         #otherwise retrun a list of nans ? This sho
```

1. Docstring

This is an example of Numpy style docstring.

```
In [ ]: """
Extract peak moments per stride for a given joint.

Parameters
-----
df: pd.DataFrame
    The DataFrame containing gait data.
joint_label: str
    The label of the joint to extract data for.
    (ex. 'Right Ankle', 'Left Knee', etc.)
direction: str
    'max' for maximum peaks, 'min' for minimum peaks.
sampling_rate: float
    The sampling rate of the data in Hz; default is 100.0 Hz.

Returns
-----
np.array
    mean of peak moments per stride
    and all peak moments per stride.
"""
```

2. Local variables & Parameters

1. More parameters to the function (ex. `df`, `sampling_rate`) instead of calling variables outside the function definition.
2. Give type hints (ex. `df: pd.DataFrame`).
3. Parameters with default values will be placed after those without default values.

```
def extract_peak_per_stride(
    df: pd.DataFrame,
    joint_label: str,
    direction: str = 'max',
    sampling_rate: float = 100.0)
```

4. Other variables are defined inside the function.
 - `moment_columns`
 - `*_frames` (ex. `lhs_frames`)
 - `bad_frame_indices`

```
In [18]: # This is how `moment_columns` is defined in the original code.
# >>> moment_columns = {
```

```
#         "Right Ankle": "Right_Ankle_Moment",
#         "Left Ankle": "Left_Ankle_Moment",
#         ...
#     }

# Then later in the code, `joint_label` is used as a key
# to get the corresponding `moment_columns` value.
# For example, if `joint_label` is "Right Ankle",
# then `moment_columns[joint_label]` is "Right_Ankle_Moment".
# >>> colname = moment_columns[joint_label]
# This is used to extract the relevant column from the DataFrame
# by the following line:
# >>> data_series = df[colname]
# which is used to extract the relevant column from the DataFrame.

# Values of `moment_columns` all contain the string: "Moment".
# Since we are now using MultiIndex columns, we can extract
# columns whose first levels contain "Moment" and `joint_label`.
joint_label = 'Right Ankle'
moment_columns = [x for x in data.columns
                  if "Moment" in x[0] and joint_label in x[0]]
moment_columns
```

Out[18]: [('Right Ankle Moment', 'LINK_MODEL_BASED', 'ORIGINAL', 'X')]

```
In [19]: # `lhs_frames`, `rhs_frames`, `lto_frames`, `rto_frames`
# are defined in the original code:
# >>> rhs_frames = to_frames(event_columns["RHS"])

# `to_frames` is a function with the following definition
# in the original code:
# >>> def to_frames(event_times, sampling_rate=100.0):
#     return ((df[col_name].dropna().astype(float) * sampling_rate)
#             .astype(int).values)

# The original function again uses `df` defined outside the function.
# `.astype(float)` ensures the data type is float
# before multiplication, but not necessary if values are multiplied
# by 100.0 (float).
# `.values` converts the Series to a numpy array.
# It's recommended to use `.to_numpy()`.

# We would define the function like this:
# 1) `df` is passed as a parameter.
# 2) `.loc` is used for indexing because we are using MultiIndex.
#    col_name is expected to be a pd.IndexSlice object.
# 3) `.ravel()` is used to convert 2D array to 1D array.
# 4) `.tolist()` is used to convert to a plain list.
def to_frames(df: pd.DataFrame, col_name: tuple | slice,
              sampling_rate: float=100.0) -> list[int]:
    """Convert gait event times from seconds to frame indices."""
    vals = df.loc[:, col_name].dropna().to_numpy().ravel()
```

```
return (vals * sampling_rate).astype(int).tolist()
```

```
In [ ]: # `bad_frame_indices` is defined in the original code as:
# >>> bad_frame_indices = set()
# >>> if event_columns["Bad"] in df.columns:
#     bad_col = df[event_columns["Bad"]].astype(str).fillna("")
#     for val in bad_col:
#         try:
#             frame = int(round(float(val) * sampling_rate))
#             bad_frame_indices.add(frame)
#         except ValueError:
#             continue

# Here, `event_columns` is defined as a dictionary elsewhere.
# (ex. {'RHS': 'RHS', 'LHS': 'LHS', ...})

# "Bad" columns contain time of 'bad' events, so they are numbers.
# We can skip the `.astype(str)` step.
# Also, see how the entire `bad_frame_indices` can be calculated
# using `to_frames`? Numbers are multiplied by the `sampling_rate`
# and converted to integers there.

# `round` is redundant here if sampling rate is 100.0 and
# the values of 'Bad' column are given to the second decimal point.
# For example, if one 'Bad' label is 8.81,
# then `float(val) * sampling_rate` -> 881.0 and int(881.0) -> 881

# This is how I would rewrite it:
# >>> try:
#     bad_frame_indices = set(
#         to_frames(
#             df,
#             pd.IndexSlice['Bad', :, :, :],
#             sampling_rate))
#     except:
#         bad_frame_indices = set()
#
# This way, even if `try` section fails,
# we will have `bad_frame_indices`, which is an empty set.
```

3. `.iloc` for indexing

Minor edit, but just to make sure. Also, `end+1` will make sure `end` index is included (if needed).

```
segment = data_series[start:end].values -> segment =
data_series.iloc[start:end+1]
```

4. Consistent return data types

If one of the two returned item is `np.ndarray`, consider making the other one to be the same.

Putting everything together...

```
In [ ]: # Here's how I would prepare:
# `df` as a parameter, and docstring added.
import numpy as np

def to_frames(df: pd.DataFrame, col_name: tuple | slice,
              sampling_rate: float = 100.0):
    """
    Convert gait event times from seconds to frame indices.

    Parameters
    -----
    df: pd.DataFrame
        The DataFrame containing gait data.
    col_name: tuple | slice
        Column header whose values will be converted (ex. 'RHS')
    sampling_rate: float
        Sampling rate of the measurement device; default is 100.0

    Returns
    -----
    list of frame indices
    """
    vals = df.loc[:, col_name].dropna().to_numpy().ravel()
    return (vals * sampling_rate).astype(int).tolist()

def extract_peak_per_stride(df: pd.DataFrame,
                             joint_label: str,
                             direction: str = 'max',
                             sampling_rate: float = 100.0):
    """
    Extract peak moments per stride for a given joint.

    Parameters
    -----
    df: pd.DataFrame
        The DataFrame containing gait data.
    joint_label: str
        The label of the joint to extract data for.
        (ex. 'Right Ankle', 'Left Knee', etc.)
    direction: str
        'max' for maximum peaks, 'min' for minimum peaks.
    sampling_rate: float
        The sampling rate of the data in Hz; default is 100.0 Hz.
```

Returns

np.array

mean of peak moments per stride and all peak moments per slide
 """

Make sure user input is valid

if direction **not in** ['max', 'min']:

raise ValueError("Direction must be either 'max' or 'min'.")

col_name = [x **for** x **in** df.columns

if "Moment" **in** x[0] **and** joint_label **in** x[0]]

data_series = df[col_name]

idx = pd.IndexSlice

if "Left" **in** joint_label:

Frame indices

HS: Heel Strike; T0: Toe Off

For example, 'RHS' is a series

(0.24, 1.31, 2.4, ..., 28.9, 29.96)

In terms of frame indices ('rhs_frames'),

it will be (24, 131, 240, ..., 2890, 2996)

given a sampling rate of 100 Hz.

 starts, ends = (to_frames(df=df,
 col_name=idx["LHS", :, :, :],
 sampling_rate=sampling_rate),

 to_frames(df,
 idx["LT0", :, :, :],
 sampling_rate))

else:

If you provide parameters in the right order,

you don't need to specify parameters.

 starts, ends = (to_frames(df,
 idx["RHS", :, :, :],
 sampling_rate),

 to_frames(df,
 idx["RT0", :, :, :],
 sampling_rate))

"""

HS can follow 'T0'

ex.

 LHS | LT0

-----+-----

0.79 | 0.36 <-- HS is later than T0

1.88 | 1.45

2.92 | 2.54

... | ...

So in this case (start, end) pairs will be formed

using the next 'T0' value: (0.79, 1.45), (1.88, 2.54), ...

Original code does it this way:

```
>>> for start in starts:
    end_candidates = ends[ends > start]
    if len(end_candidates) == 0:
        continue
    end = end_candidates[0]
```

Instead, we can just skip the first 'end' value
if the first 'start' is later than the first 'end'.
"""

```
if starts[0] > ends[0]:
    ends = ends[1:]
```

```
try:
    bad_frame_indices = set(
        to_frames(df,
                   pd.IndexSlice['Bad', :, :, :],
                   sampling_rate))
```

```
except:
    bad_frame_indices = set()
```

```
peak_values = []
```

```
# Using `zip()` can skip finding `end` by
# creating another variable: `end_candidates`.
# `zip()` stops at the shortest input,
# so no IndexError will occur.
```

```
for start, end in zip(starts, ends):
    # Skip short segments (less than 5 frames).
    # This replaces `if len(segment) < 5: continue`
    if end - start < 5:
        continue
    # If 'bad' frame exists in this cycle, skip it.
    cycle_frame_range = set(range(start, end))
    # Original logic is double negative (`not isdisjoint`)
    # Alternatively, use `&` operator or `.intersection` method
    # to make a set whose elements are both present in
    # `cycle_frame_range` and `bad_frame_indices`
    # and if that set has length > 0, `continue`
    if (cycle_frame_range & bad_frame_indices):
        continue
```

```
segment = data_series.iloc[start:end]
peak_val = segment.max() if direction == 'max' \
    else segment.min()
peak_values.append(peak_val)
```

```
# Originally, a numpy array and
# a list (peak_values) were returned.
# Be more consistent by returning a numpy array for both.
# Along with np.nan, return an empty numpy array,
# not an empty list.
if peak_values:
```

```

        return np.mean(peak_values), np.array(peak_values)
    else:
        return np.nan, np.array([])

```

Let's check if the function works as intended. Let's check if it can extract peak moment per stride using the **Right Knee** moments. We will be finding the **maximum** of segments.

Here are the times (second) of the right heel strikes (RHS) and right toe offs (RTO).

RHS	RTO
0.24	0.92
1.31	2.02
...	...
28.9	29.58
29.96	NA

← This will not be included

The first segment will be from 0.24s - 0.92s, or 24th frame to 92th frame.

```

In [21]: # Test: Right Knee Moment
pk_mean, pks = extract_peak_per_stride(data, "Right Knee", 'max')
# This is the max of the first segment
pks[0]

```

```

Out[21]: array([0.56576])

```

```

In [22]: # Check the segment:
segment = data.loc[24:93,
                  ('Right Knee Moment',
                   'LINK_MODEL_BASED', 'ORIGINAL', 'X')]
# Maximum of the segment: is it the same as `pks[0]`?
max(segment)

```

```

Out[22]: 0.56576

```

```

In [23]: # Test 2: Left Hip Moment
pk_mean_lhip, pks_lhip = extract_peak_per_stride(
    data, "Left Hip", 'min')
# Let's try the second segment (1.88 ~ 2.54s)
segment_lhip = data.loc[131:255,
                       ('Left Hip Moment',

```

```

                                'LINK_MODEL_BASED', 'ORIGINAL', 'X'])
# Is it `True`?
pks_lhip[1] == min(segment_lhip)

```

```
Out[23]: array([ True])
```

4. More data manipulation

This section is really not relevant to most...

Stance velocities (lines 216-251)

```

In [ ]: # ---- previous info needed for demo purpose ----
sampling_rate = 100.0
# This is an example of using `map()` to apply `to_frames`
# to get all four frame lists at once.
# This is rather advanced usage; you can just call
# `to_frames` four times separately.
rhs_frames, rto_frames, lhs_frames, lto_frames = \
    map(lambda x: to_frames(data,
                             pd.IndexSlice[x, :, :, :],
                             sampling_rate=sampling_rate),
         ['RHS', 'RTO', 'LHS', 'LTO'])
# A more accessible(?) way using the list comprehension:
# >>> rhs_frames, rto_frames, lhs_frames, lto_frames = \
#     [to_frames(data,
#                 pd.IndexSlice[event, :, :, :],
#                 sampling_rate=sampling_rate)
#       for event in ['RHS', 'RTO', 'LHS', 'LTO']]
# -----

# ---- cleaning codes ----
# Here we can prepare a small function that returns a list of tuples:
# (start, end)
# This can also be used inside `extract_peak_per_stride`

def return_windows(hs_frames: list, to_frames: list) -> list:
    """
    From a HS (heel strike) and a T0 (toe off) frame series,
    make a list of tuples of gait start and end frame.

    Parameters
    -----
    hs_frames: list
        List of heel strike frames; output of `to_frames()`
    to_frames: list
        List of toe off frames; output of `to_frames()`

    Returns
    -----

```

```

list of tuples
"""
# If the first 'start' is later than the potential 'first' end,
# take the next end to pair with this first start.
if hs_frames[0] > to_frames[0]:
    return list(zip(hs_frames, to_frames[1:]))
return list(zip(hs_frames, to_frames))

# For example,
left_frames = return_windows(lhs_frames, lto_frames)
print("First five left gait cycles (start, end):")
for start, end in left_frames[0:5]:
    print(f"Start: {start}, End: {end}")

```

```

First five left gait cycles (start, end):
Start: 79, End: 145
Start: 188, End: 254
Start: 292, End: 357
Start: 398, End: 463
Start: 508, End: 572

```

```

In [25]: # Then define another wrapper function
def get_stance_velocities(df: pd.DataFrame,
                          side: str = "Left",
                          sampling_rate: float = 100) -> list:
    """
    Parameters
    -----
    df: pd.DataFrame
        Gait data with CGVel average columns populated.
    side: str
        Which leg data to analyze; default is "Left"
    sampling_rate: float
        The sampling rate of the data in Hz; default is 100.0 Hz.

    Returns
    -----
    middle_avgs: list
        List of stance velocities
    """
    middle_avgs = list()

    if side == "Left":
        gait_frames = return_windows(
            to_frames(df, pd.IndexSlice['LHS', :, :, :],
                      sampling_rate),
            to_frames(df, pd.IndexSlice['LTO', :, :, :],
                      sampling_rate))
        col_label = "LFT"
    else:
        gait_frames = return_windows(
            to_frames(df, pd.IndexSlice['RHS', :, :, :],
                      sampling_rate),

```

```

        to_frames(df, pd.IndexSlice['RTO', :, :, :],
                  sampling_rate))
    col_label = "RFT"

    try:
        bad_frame_indices = set(
            to_frames(data, pd.IndexSlice['Bad', :, :, :],
                    sampling_rate))
    except:
        bad_frame_indices = set()

    for start, end in gait_frames:
        # `cycle_range` is a set
        cycle_range = set(range(start, end))
        # Both `cycle_range` and `bad_frame_indices` are sets
        # so you can check if `cycle_range.isdisjoint` with
        # `bad_frame_indices`.
        # If they are disjoint (meaning no bad frames in this cycle),
        # then proceed.
        if cycle_range.isdisjoint(bad_frame_indices):
            # ex. ("CGVel", "KINETIC_KINEMATIC", "LFT", "Avg")
            avg_slice = pd.IndexSlice["CGVel", :, col_label, "Avg"]
            cgvel_segment = df.loc[start:end, avg_slice].to_numpy()
            backward_values = cgvel_segment[cgvel_segment < 0]
            if len(backward_values) >= 5:
                n = len(backward_values)
                start_idx = int(0.1*n) # 10% index
                end_idx = int(0.6*n)   # 60% index
                middle_avgs.append(
                    backward_values[start_idx:end_idx].mean())

    return middle_avgs

```

```

In [26]: # How to use it:
left_stance_velocities = get_stance_velocities(data, "Left")
right_stance_velocities = get_stance_velocities(data, "Right")

```

Ensemble averaging (lines 295-393)

```

In [27]: # Another helper function:
def make_segment(df: pd.DataFrame,
                joint_label: str,
                start: int,
                end: int) -> np.ndarray:
    """
    Using start and end frame indices,
    extract the segment of moment data for the given joint.
    """
    col_name = [x for x in df.columns
                if "Moment" in x[0] and joint_label in x[0]]
    data_series = df[col_name]

```

```
return data_series.iloc[start:end].to_numpy().ravel()
```

In [28]: **from** scipy.interpolate **import** interp1d

```
def interpolate_segments(df: pd.DataFrame,
                        joint_label: str,
                        sampling_rate: float = 100.0):

    time_seconds = np.arange(len(df)) / sampling_rate
    n_points = 101

    bad_segment_count = 0
    all_cycles = list()
    impulses = list()
    impulses_ext = list()
    impulses_flex = list()

    if "Left" in joint_label:
        gait_frames = return_windows(
            to_frames(df,
                      pd.IndexSlice['LHS', :, :, :],
                      sampling_rate),
            to_frames(df,
                      pd.IndexSlice['LTO', :, :, :],
                      sampling_rate))
    else:
        gait_frames = return_windows(
            to_frames(df,
                      pd.IndexSlice['RHS', :, :, :],
                      sampling_rate),
            to_frames(df,
                      pd.IndexSlice['RTO', :, :, :],
                      sampling_rate))

    try:
        bad_frame_indices = set(
            to_frames(data,
                      pd.IndexSlice['Bad', :, :, :],
                      sampling_rate))
    except:
        bad_frame_indices = set()

    for start, end in gait_frames:
        cycle_frame_range = set(range(start, end))

        if (cycle_frame_range & bad_frame_indices):
            bad_segment_count += 1
            continue

        segment = make_segment(df, joint_label, start, end)
        time_segment = time_seconds[start:end]
```

```

if len(segment) < 5:
    continue

"""
Original lines 343-347
Interpolate to fixed number of points
`np.linspace(0, 1, len(segment))` creates
an array of evenly spaced values from 0 to 1
with the same length as `segment`.
This is used as the x-coordinates for the original data points
`interp1d` creates a linear interpolator function
based on these original data points.
Then, `interpolator(np.linspace(0, 1, n_points))`
generates `n_points` (101) evenly spaced values
from 0 to 1 and uses the interpolator to get
the corresponding y-values (interpolated segment values).
"""
x_old = np.linspace(0, 1, len(segment))
interpolator = interp1d(x_old, segment, kind='linear')
normalized = interpolator(np.linspace(0, 1, n_points))
all_cycles.append(normalized)

# Original lines 349-366
if "Ankle" in joint_label:
    choose = segment < 0
    impulses.append(
        # `np.trapezoid` computes the integral
        # using the trapezoidal rule.
        np.trapezoid(segment[choose],
                      time_segment[choose]) \
        if np.any(choose) else np.nan
    )
elif "Knee" in joint_label:
    choose = segment > 0
    impulses.append(
        np.trapezoid(segment[choose],
                      time_segment[choose]) \
        if np.any(choose) else np.nan
    )
elif "Hip" in joint_label:
    choose_ext = segment < 0
    choose_flex = segment > 0
    if np.any(choose_ext):
        impulses_ext.append(
            np.trapezoid(segment[choose_ext],
                          time_segment[choose_ext]))
    if np.any(choose_flex):
        impulses_flex.append(
            np.trapezoid(segment[choose_flex],
                          time_segment[choose_flex]))

```

```

return {"bad_segment_counts": bad_segment_count,
        "all_cycles": np.asarray(all_cycles),
        "impulses": impulses,
        "impulses_ext": impulses_ext,
        "impulses_flex": impulses_flex}

```

5. Plotting using a for loop

```

In [29]: import matplotlib.pyplot as plt

metadata = {
    "Subject": "Shefaali",
    "Session": "",
    "Condition": "",
    "Walk": "BaselineWalk",
    "GaitSpeed": "1.2"
}

# Here subplot locations (row, column) are paired with
# `joint_label`. This can be one good way to designate
# which variable will be plotted where.
subplot_layout = [
    ("Left Ankle", 0, 0), ("Right Ankle", 0, 1),
    ("Left Knee", 1, 0),  ("Right Knee", 1, 1),
    ("Left Hip", 2, 0),   ("Right Hip", 2, 1)
]

# `layout='constrained'` is useful when you have subplots
# and want to adjust space between them so that no overlap
# of axis labels happen.
fig, axes = plt.subplots(3, 2, figsize=(12, 10),
                        sharex=True, layout='constrained')
# Unpacking the three elements (ex. "Left Ankle", 0, 0)
# Here's one good use of using for loop to iterate over
# variables you want to plot in one figure.
for joint_label, row, col in subplot_layout:
    # Remember, `data` is the DataFrame we read.
    # `interpolate_segments` is the function defined in Section 4
    # It basically time normalizes data, so that moments at
    # different joint can be plotted using the same scale.
    intp_results = interpolate_segments(data, joint_label)
    ax = axes[row, col]

    all_cycles = intp_results['all_cycles']
    x_new = np.arange(0, 101)
    if len(all_cycles):
        mean_cycle = all_cycles.mean(axis=0)
        std_cycle = all_cycles.std(axis=0)
        ax.plot(x_new, mean_cycle, label='Mean')
        ax.fill_between(x_new, mean_cycle - std_cycle,

```



```

mean_cycle + std_cycle, alpha=0.3)

ax.set_title(joint_label)
ax.set_ylabel("Moment (Nm/kg*m)")
ax.grid(False)
ax.axhline(0, c='k', linestyle='--')

# As x-axis is shared (`sharex=True`),
# only set the last x-axis labels.
axes[2, 0].set_xlabel("Percent Stance Phase")
axes[2, 1].set_xlabel("Percent Stance Phase")
# Figure-level title.
fig.suptitle(f"{metadata['Subject']} "
             f"{metadata['Session']} "
             f"{metadata['Walk']} "
             f"{metadata['Condition']} ",
             fontsize=16, y=1.02)
# Uncomment and run the line below to save this figure.
# fig.savefig("Some_figure.png", dpi=300, bbox_inches='tight')

```

Out[29]: Text(0.5, 1.02, 'Shefaali BaselineWalk ')

